Checkers AI Agent

Jack Underhill, Chandler Guthrie, Matthew Covey, Jamil Staten

CPTS 440

4/29

What is our project?

- Build an AI agent that can play checkers
- Use legal game rules to generate valid moves
- Select the best move from many possible future states
- Provide a playable interface for testing and demonstration

Why is this an AI problem?

- The agent must make decisions under competition
- It must reason about future moves, not just the current board
- It needs an evaluation strategy when it cannot search the full game tree
- It is a classic adversarial search problem

How we modeled the problem

- Simplified American Checkers
  - 6 by 6 Checkerboard
  - Kings act like Mans that can go Backwards
- AI vs Player Game
- Web Client and Server

Important game logic

- The board is a 6x6 grid scored as a 2D list
- Pieces are represented as single characters: r/R for red man/king, b/B for black man/king
- Move generation checks all diagonal directions per piece; men move forward only, kings move in all four directions
- Capture priority is enforced
- Multi jump captures are resolved recursively until no further capture is available
- A man is promoted to king when reaching the opponents closest rank
- A player loses when they have no pieces remaining or no legal moves

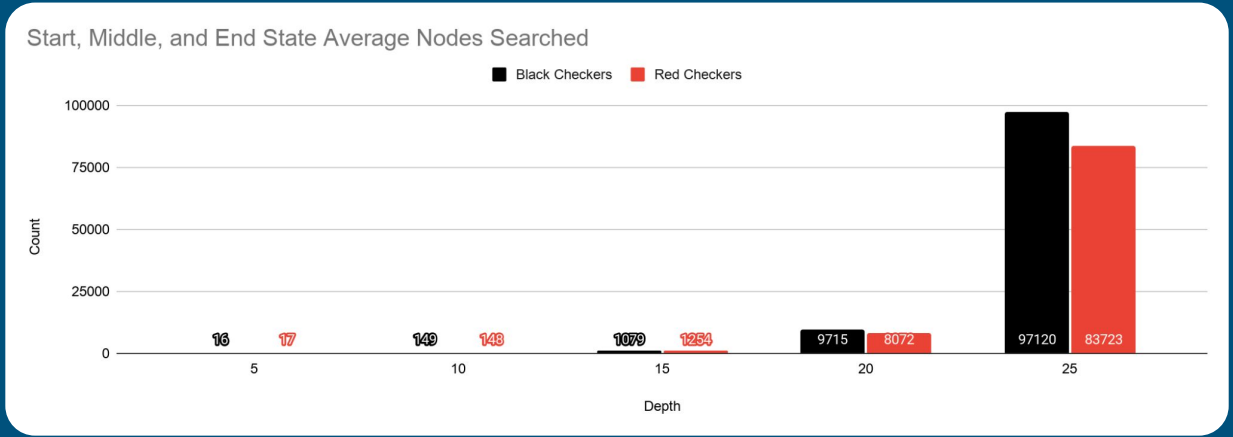
Core AI method

- Minimax With Alpha Beta Pruning
- Evaluate Early Using Depth Constraint
- Similarity States For More Pruning
- Bits to Represent the Board
  - Bitwise Calculations for Speed Complexity
  - Bits vs Array for Space Complexity

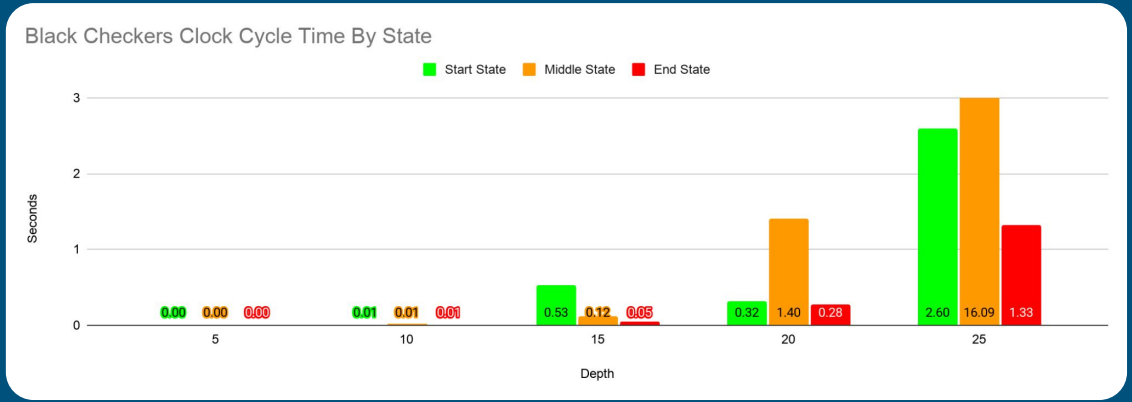
How our agent evaluates a board Part 1

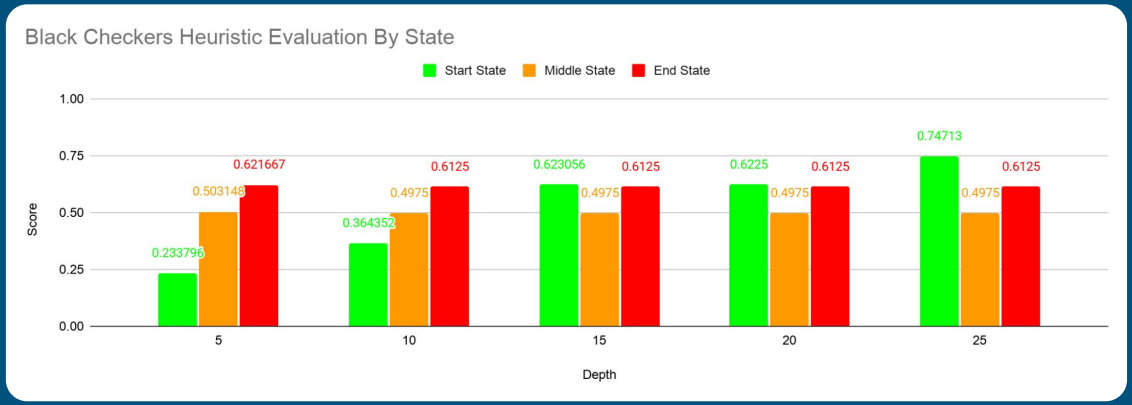
- Weighted Features
  - Admissible Heuristic using a Weighted Linear Function
- Checkers Count
  - Checkers captured
  - Max’s Alive Checkers
    - Kings worth more than man
  - Stop opponent man’s from becoming king
- Max’s Man distance to becoming a king

How our agent evaluates a board Part 2



How our agent evaluates a board Part 3





Main components

- Board data structure - tracks positions, colors, types, and enforces rules
- State generator - produce states from given position covering moves
- Minimax with alpha/beta pruning - evaluates future game states to pick the best move to commit to on the board
- Heuristic function - scores board states based on piece count advantage, king promotions, and piece advancement against opponent
- DFS with limited deepening - controls search depth to keep agent responsive during gameplay instead of searching entire game tree
- CLI/GUI - Terminal and visual web-based interface to play against AI

How the work was divided

- Jamil Staten: Board data structure: designed the 6x6 board representation, piece encoding, and initial game state
- Matthew Covey: State generation; implemented legal move enumeration, capture priority, and successor state application
- Chandler Guthrie: AI algorithm; build the minimax search with alpha beta pruning, depth limiting, and the admissible heuristic
- Jack Underhill: Integration; connected all components into the working system, build the Reac/FastAPI web interface

What we achieved

- Fully functional 6x6 checkers game in Python
- Implemented board data structure tracking of positions, pieces, moves, etc
- Developed a state generator producing valid moves from board positions
- Minimax AI agent with alpha/beta pruning and an admissible heuristic
- Heuristic recognizes penalizing, capturing, and advancing pieces
- Delivered a CLI and GUI interface for humans to play against AI
- Agent has an adjustable difficulty via heuristic tuning

Limitations & Future Improvements

- Mapping from Array To Bit Form
  - Very Costly
  - Transform Game Logic To Use Bits Form
- Better Features
  - Ending Condition (player vs opponent checker distances)
  - Similar Board Pattern Checking
- Toggle For International Rules
  - 10 by 10 Board Size
  - Enforce rules (Backward Capture, Capture Only Choices, etc.)

Demo

https://checkers-ai-agent.netlify.app/

Conclusion

- Successfully constructed a functional checkers AI opponent
- Implemented all core components such as a game engine, state generator, minimax search, and a heuristic function
- Developed a GUI web interface for a visual and interactive playful experience
- Heuristic agent can be adjusted in difficulty for different players
- Obtained knowledge with adversarial search and AI decision making
- A future implementation/improvement could be a 10x10 board, a timer to make your move, or better heuristics for a more advanced play